

Student name: Deepak Raj

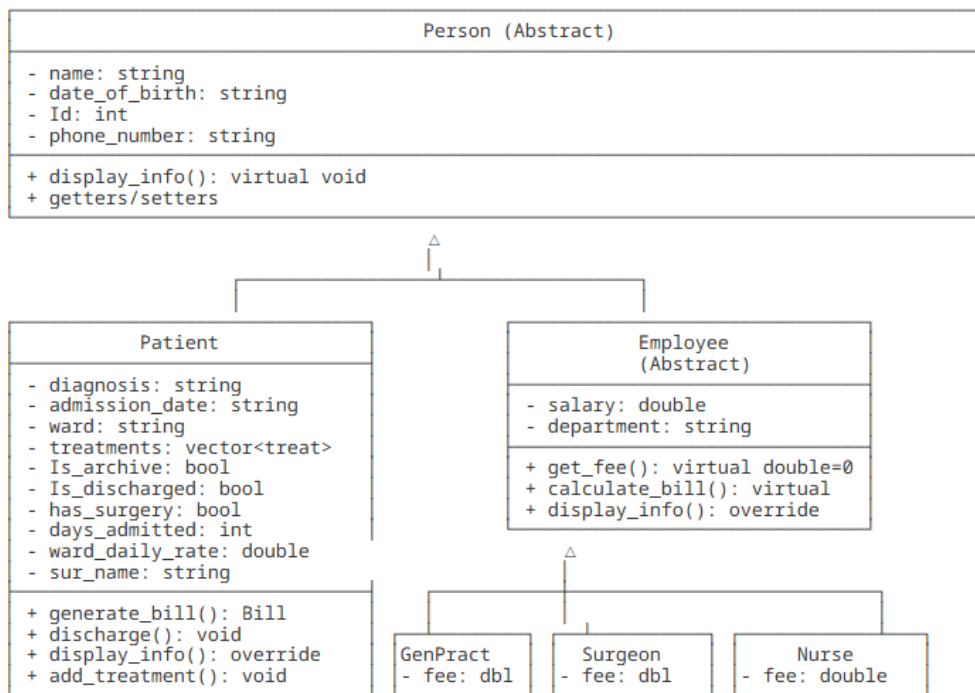
ERP: 32425

Instructor: Sir Behraj

Subject: Object Oriented Programming

Class Diagram

The following diagram outlines the inheritance structure of the system, highlighting the relationship between Persons, Patients, and Staff.



Core structures for managing billing, appointments, and ward capacity.

treatment
- t_name: string - cost: double - staff_name: string
+ getters/setters

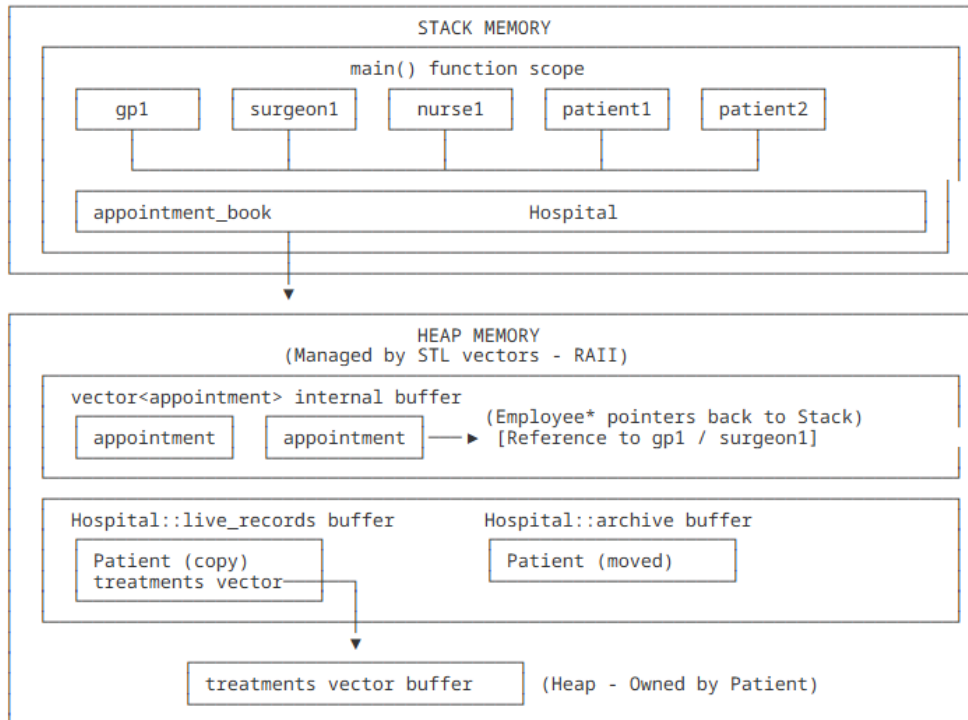
Bill
- treatment_cost: double - ward_fee: double - admin_fee: double
+ total(): double + operator==, +, <<

Ward
- name: string - capacity: int - patients: vector<Patient>
+ occupancy_percentage(): dbl

Hospital
- live_records: vector<Pat> - archive: vector<Pat>
+ admit/discharge() + wardRevenue(): double

Memory management map

A visualization of Stack vs. Heap allocation. The system utilizes **RAII (Resource Acquisition Is Initialization)** via STL containers, ensuring no manual memory leaks.



Design Decision Justifications

1. Inheritance Hierarchy

Used an abstract `Person` base class extended by `Patient` and `Employee`, with further specialization (General Practitioner, Surgeon, Nurse).

→ Supports **polymorphism** and follows the **Open/Closed Principle** for easy extension.

2. Raw Pointers in Appointment

`Appointment` stores `Employee*` (non-owning).

→ Ensures **no slicing**, avoids overhead, and maintains clear ownership since `main()` manages object lifetimes.

3. Ward Hierarchy with Virtual Functions

Base `Ward` class with derived types overriding `add_patient()`.

→ Enables **polymorphic admission rules** and keeps logic **encapsulated and extensible**.

4. **Operator Overloading in Bill**

Implemented +, ==, and <<.

→ Provides **natural syntax**, **STL compatibility**, and easy output handling.

5. **Move Semantics in Discharge**

Used std::move when transferring patients to archive.

→ Improves **performance** and avoids unnecessary copying.

6. **Static ReportingSystem**

Reporting functions are static.

→ Follows **utility class pattern** (stateless, no instantiation needed).

7. **STL Algorithms Usage**

Used algorithms like copy_if, sort, accumulate.

→ Improves **readability**, **reliability**, and allows **better optimization**.

8. **Deep Copy in Patient**

Default copy constructor copies vectors deeply.

→ Ensures **independent objects** and proper **value semantics**.

9. **No Manual Memory Management**

Relied on stack objects and STL containers.

→ Ensures **RAII**, **no memory leaks**, and **exception safety**.

10. **Ward Comparison Operators**

Implemented <, >, == based on patient count.

→ Enables **sorting** and meaningful comparison of ward occupancy.

11. **Const-Correctness**

generate_bill() marked const.

→ Reflects correct semantics, though minor design refinement may remove need for const_cast.

Summary

The design emphasizes:

- **Polymorphism** (staff roles, ward rules)
- **RAII** (safe memory management)
- **STL usage** (clean and efficient data handling)
- **Operator overloading** (intuitive interfaces)
- **Clear ownership** (raw pointers only for non-owning references)